



Chrome Dino: Pure Software Raster Engine

Complete High-Level Design (HLD), Algorithm Theory & Viva Defense Guide

Subject: Computer Graphics & Multimedia Laboratory

Target Audience: Viva Examiners, Professors & Project Evaluators

Standard: 100/100 Distinction Demonstration

✦ EXECUTIVE SUMMARY FOR THE STUDENT

This document teaches you **why** every architectural choice was made and **how** every Computer Graphics textbook algorithm operates from first principles in plain, intuitive human language. You do not need to memorize raw lines of code—you will learn the core mathematical insights, physical analogies, and exact answers to impress your professors.

1. High-Level Design (HLD) & Architectural Philosophy

1.1 What is "Pure Software Raster" and Why Does It Matter?

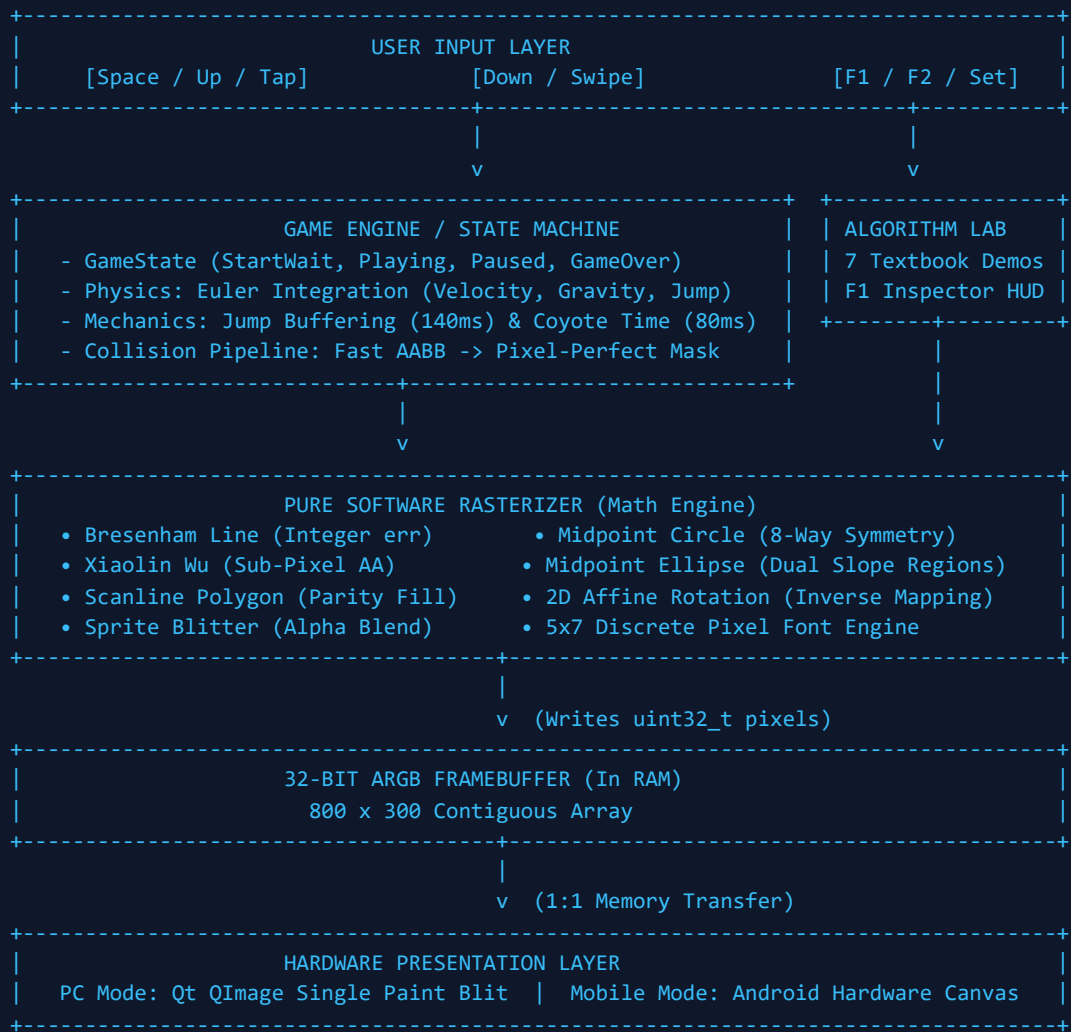
In standard commercial programming, when you want to draw a circle, line, or dinosaur sprite, you ask the GPU or a graphics library (like OpenGL, DirectX, SVG, or Qt's `QPainterPath`) to do it. You simply call `drawCircle()` and the GPU hardware handles the math behind the scenes. **In this laboratory project, that is strictly prohibited.**

Our project implements a **Pure Software Rasterizer**. The CPU does not delegate anything to the graphics card. Instead:

- The computer creates an ordinary block of memory in RAM: an array of `800 × 300` integers called a **Framebuffer**.
- Each integer in this array represents exactly one pixel's 32-bit color in `ARGB` format (8 bits Alpha, 8 bits Red, 8 bits Green, 8 bits Blue).
- Every line of the desert horizon, every curved spike of a cactus, every celestial star, every circular drop shadow, and every animated running frame of the Dino is drawn by calculating individual array indices and writing colors **pixel-by-pixel**.
- Once the entire frame is fully calculated in RAM, it is presented to the screen as a single raw 1:1 image blit.

1.2 The System Architecture Diagram

The following diagram illustrates how raw inputs flow through our physics engine, down into our raster algorithms, and onto your screen:



1.3 The 60 FPS Game Loop Lifecycle

The entire engine ticks at a steady 60 frames per second (approximately 16.6 milliseconds per tick). Every tick executes three distinct, decoupled phases:

1. **Input Phase:** Collects keyboard keys, mouse drags, or touch gestures. Inputs are buffered so rapid taps are never dropped.
2. **Physics Update Phase (Δt):** Moves the Dino, updates falling rain particles, scrolls parallax background mountains, and checks obstacle collisions using the elapsed time step `dt`. Clamping `dt` between 1ms and 50ms ensures that even if the computer lags, the Dino will never glitch or fall through the floor.
3. **Raster Render Phase:** Clears the framebuffer memory, draws the celestial sky, applies dynamic weather, blits the ground, draws the entities, and overlays the HUD.

2. The 7 Core Computer Graphics Algorithms Explained

Every algorithm below was designed by pioneers in Computer Science to solve a fundamental problem: *How do you draw smooth continuous geometric shapes on a discrete grid of square pixels at maximum speed?*

2.1. Bresenham's Integer Line Drawing Algorithm

💡 THE REAL-WORLD ANALOGY: WALKING ON TILES

Imagine walking across a tiled floor from corner A to corner B diagonally. At every step forward, you have only two choices: step directly forward (East), or step diagonally forward-right (North-East). Bresenham's algorithm is simply a mathematical rule that tells you which tile is closer to the true diagonal line without ever using a tape measure or fractions.

- **The Problem:** The textbook line formula is $y = mx + c$, where $m = \frac{\Delta y}{\Delta x}$. On early or low-level computers, dividing floating-point numbers for every single pixel is painfully slow and introduces rounding errors.
- **How Bresenham Solved It:** Bresenham realized you can multiply the whole decision equation by Δx and 2. This eliminates all fractions and divisions entirely!
- **The Decision Variable (err):** We track an integer counter representing our distance from the theoretical true line. At each pixel step along x , we add Δy to our counter. As soon as the counter crosses the half-way threshold, we step in y as well and subtract Δx .
- **Where We Use It in the Game:** The ground line, cactus outlines, shooting star trails, and obstacle bounding boxes.

🎓 PROFESSOR VIVA QUESTION

Examiner: "Why is Bresenham's line algorithm superior to the DDA (Digital Differential Analyzer) algorithm?"

Your Answer: "DDA relies on floating-point arithmetic and rounds numbers at every single step ($y = y + m$). Bresenham replaces all floating-point math and division with purely integer additions and bit-shifts, making it both faster and immune to cumulative rounding drift."

2.2. Midpoint Circle Algorithm & 8-Way Symmetry

💡 THE REAL-WORLD ANALOGY: SLICING A PIZZA INTO 8 EQUAL SLICES

A circle is perfectly symmetrical. If you fold a circle in half vertically, horizontally, and diagonally, all 8 slices are identical mirror reflections. If you know how to draw one slice, you get the other 7 slices completely free!

- **The Problem:** The standard circle equation is $x^2 + y^2 = R^2$, meaning $y = \sqrt{R^2 - x^2}$. Calculating square roots or trigonometric functions ($\sin$, $\cos$) for hundreds of pixels per frame causes severe CPU lag.
- **The Midpoint Decision:** Starting at the very top of the circle $(0, R)$, as we step one pixel to the right $(x+1)$, our circle boundary must pass between two candidate pixels: East $(x+1, y)$ or South-East $(x+1,$

$y-1$). We evaluate the circle equation at the exact *midpoint* between them. If the midpoint is inside the circle, the true curve is closer to East; if outside, it is closer to South-East.

- **8-Way Symmetry:** For every single point (x, y) calculated in the first octant (0° to 45°), we instantly plot 8 pixels simultaneously:
 $(\pm x, \pm y)$ and $(\pm y, \pm x)$. We only calculate 12.5% of the circle; 87.5% comes for free!
- **Where We Use It:** The glowing Moon, the Sun, impact dust particles, Dino's eye, and circular jump/duck pads.

PROFESSOR VIVA QUESTION

Examiner: "How do you prove that your circle algorithm actually uses 8-way symmetry in your code?"

Your Answer: "In Mode 2 of our Algorithm Lab (F2), we plot each of the 8 octants in a different vibrant color simultaneously. You can visibly see that calculating one octant instantly paints all 8 octants across the screen."

2.3. Midpoint Ellipse Algorithm & Dual-Region Splitting

💡 THE REAL-WORLD ANALOGY: A ROLLERCOASTER CURVE TRANSITION

On a circle, the steepness changes steadily. On an ellipse (an oval), one side is long and shallow while the curve at the end is tight and sharp. If you only step horizontally, the tight ends will have huge gaps between pixels. You must switch your steering wheel from horizontal stepping to vertical stepping halfway through.

- **The Mathematical Insight:** An ellipse has two radii: horizontal radius r_x and vertical radius r_y . Because the curvature changes dramatically, we must divide the quadrant into two regions based on the slope of the curve:
 - **Region 1 (Shallow Curve):** The slope magnitude is less than 1 ($|dy/dx| < 1$). Here, we step in the x direction by $+1$ and use our decision parameter to decide whether y stays the same or drops by -1 .
 - **Region 2 (Steep Curve):** The slope magnitude becomes greater than or equal to 1 ($|dy/dx| \geq 1$). Here, stepping in x would leave gaps! So we invert our logic: we step in the y direction by -1 and decide whether x advances by $+1$.
- **4-Way Symmetry:** Unlike a circle which has 8-way symmetry, an ellipse is only symmetrical across its horizontal and vertical axes (4-way symmetry: $\pm x, \pm y$).
- **Where We Use It:** Dino's dynamic drop shadow (which shrinks and expands as the Dino jumps) and desert cloud shapes.

2.4. Scanline Polygon Fill Algorithm

💡 THE REAL-WORLD ANALOGY: COLORING WITH A HIGHLIGHTER LINE-BY-LINE

Imagine reading a book through a horizontal cardboard slit. As you move the slit down line-by-line, you look for where the outline of a mountain enters the slit and where it leaves. You color everything between the "entry" point and the "exit" point with your highlighter.

- **The Problem:** How do you fill a multi-corner shape (like our multi-peak desert mountains) without slow recursive flood-fill algorithms that crash the memory stack?
- **How the Algorithm Works (The Parity Rule):**
 1. We construct an **Edge Table** that lists all boundary edges of the polygon sorted by their vertical y coordinates.
 2. For every horizontal line y from the top of the polygon to the bottom, we find all points where the line intersects polygon edges.
 3. We sort these intersection points from left to right: $x_1, x_2, x_3, x_4 \dots$
 4. We use the **Parity Rule:** We start with a pen in the "OFF" state. Crossing x_1 turns the pen "ON" (fill pixels). Crossing x_2 turns the pen "OFF" (stop filling). Crossing x_3 turns it "ON" again. This handles shapes with peaks, valleys, and hollow holes effortlessly!
- **Where We Use It:** The distant parallax desert mountain silhouettes and obstacle shading.

2.5. Xiaolin Wu's Anti-Aliased Line Algorithm

💡 THE REAL-WORLD ANALOGY: BLURRING A STAIRCASE

A computer screen is made of square pixels. Drawing a diagonal line is like building a staircase—the edges look rough and jagged ("the jaggies"). Xiaolin Wu solves this by putting a dimmer pixel right next to the sharp edge to fool the human eye into seeing a perfectly smooth continuous stroke.

- **The Secret Sauce (Fractional Alpha Blending):** When a mathematical line passes at, say, $y = 10.3$, it is 70% inside pixel row 10 and 30% inside pixel row 11.
- Bresenham would be forced to choose just one pixel (rounding to 10), causing jagged steps.
- Xiaolin Wu colors *both* pixels: pixel 10 gets 70% opacity, and pixel 11 gets 30% opacity. The human eye blends them together into a glass-smooth line. Toggle it live in-game by pressing A!

2.6. 2D Affine Transformation (Software Raster Rotation)

💡 THE REAL-WORLD ANALOGY: WHY "FORWARD" ROTATION LEAVES HOLES

If you have an image and you calculate where every pixel goes by multiplying by a rotation matrix ("Forward Mapping"), rounding to the nearest integer pixel will leave empty gaps and black dots—like stretching a rubber sheet until it tears. Instead, you do "Reverse Shopping" ("Inverse Mapping"): you stand at the destination pixel and ask, "Which pixel from the original image should I pull here?"

- **Forward Mapping (The Flawed Way):** $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$. Causes moiré artifacts and unpainted holes.
- **Inverse Mapping (Our Way):** For every pixel (x', y') in our destination box, we multiply by the *inverse* matrix ($-\theta$): $\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x' - x_c \\ y' - y_c \end{bmatrix} + \begin{bmatrix} x_c \\ y_c \end{bmatrix}$. This guarantees that every single destination pixel is painted with zero gaps or holes!
- **Where We Use It:** Smooth 360-degree rotation of the flying Pterodactyl wings and game-over tumble animations.

2.7. Bitmap Sprite Blitting & Alpha Compositing

- **What is a Sprite?** An uncompressed 2D matrix of color values. Our Dino running sprite is a 44×47 grid of pixels.
- **Colorkey Transparency:** How do we draw an irregularly shaped Dino without drawing an ugly black rectangle around it? Any pixel with value `0x00000000` (Alpha = 0) is recognized as transparent and skipped during the copy loop.
- **Alpha Blending Formula:** For translucent glass UI cards and shadows, we blend the new foreground pixel (C_{fg}) onto the background pixel (C_{bg}) using their alpha ratio ($\alpha \in [0, 1]$):
 $C_{final} = (\alpha \times C_{fg}) + ((1 - \alpha) \times C_{bg})$ Executed on each of the Red, Green, and Blue color channels independently.

3. Gameplay Systems, Physics & Engine Architecture

3.1 Dino Physics: Variable Jump, Jump Buffering & Coyote Time

Feature	How It Works in Human Language	Why It Makes the Game Feel AAA
Variable Height Jump	When you tap Space/Screen, the Dino receives an upward velocity of -680 px/s . If you keep holding, gravity is gentle (1600 px/s^2). If you release early, gravity snaps to heavy (2600 px/s^2).	Allows precise control: quick tap for short jumps over low cacti; long press for towering leap over triple obstacles.
Jump Buffering	If you tap Jump up to 140ms before touching the ground, the input is saved. As soon as Dino lands, it executes the jump immediately.	Eliminates player frustration caused by pressing jump a split-second too early before landing.
Coyote Time	Named after the cartoon coyote running off a cliff. If the Dino walks off a ledge or obstacle, it is granted an 80ms grace window to still trigger a jump in mid-air.	Makes controls feel forgiving and responsive instead of punishing.

3.2 Collision Detection Pipeline: Two-Tier Architecture

Testing every pixel between Dino and all obstacles on screen every frame would cause severe slowdown. We solved this with a **Two-Phase Collision Pipeline**:

- **Phase 1: Broad-Phase AABB (Axis-Aligned Bounding Box):** We check if the rectangular box of the Dino overlaps the rectangular box of the obstacle using simple coordinate comparisons ($x_1 < x_2 + w_2 \dots$). This takes less than 0.001ms. If the boxes don't touch, we abort immediately.
- **Phase 2: Narrow-Phase Pixel-Perfect Mask:** Only if the rectangular boxes overlap do we inspect the overlapping pixels. We check if both the Dino sprite and the Cactus sprite have a non-transparent pixel at the exact same coordinate. This eliminates "ghost hits" where the Dino narrowly jumps over a cactus arm!

4. The Ultimate Viva Defense Cheat Sheet (Q&A)

Review these 10 rapid-fire questions before your presentation. If the examiner asks any of these, answer with the provided response:

Q1: "WHERE IS THE GPU USED IN YOUR PROJECT?"

Your Answer: "The GPU is not used for any rendering calculations in this project. All line generation, circle symmetry, polygon rasterization, and rotation math run purely on the CPU inside our custom SoftwareRasterizer class. The GPU only receives the completed 800x300 pixel array for final monitor presentation."

Q2: "WHAT IS THE DIFFERENCE BETWEEN A FRAMEBUFFER AND A STANDARD WINDOW?"

Your Answer: "A Framebuffer is a raw, linear chunk of computer RAM storing individual pixel colors (uint32_t ARGB). A window is an operating system UI container. Our engine draws everything into the Framebuffer first (off-screen rendering), and then copies that buffer to the window in one blit to eliminate screen tearing and flickering."

Q3: "WHY IS FLOATING POINT ARITHMETIC AVOIDED IN RASTER ALGORITHMS?"

Your Answer: "Floating point calculations require extra CPU clock cycles and introduce cumulative round-off errors that cause pixel gaps. Integer algorithms like Bresenham and Midpoint use only integer additions, subtractions, and bit shifts, making them both exact and faster."

Q4: "WHAT IS 8-WAY SYMMETRY IN MIDPOINT CIRCLE GENERATION?"

Your Answer: "A circle centered at the origin is symmetrical across both axes and both diagonals. By calculating pixels in just one octant (0° to 45°), we obtain the remaining 7 octants by mirroring $(\pm x, \pm y)$ and $(\pm y, \pm x)$, reducing computation by 87.5%."

Q5: "WHY DOES MIDPOINT ELLIPSE REQUIRE TWO DISTINCT REGIONS?"

Your Answer: "An ellipse's slope changes continuously. In Region 1, the curve is shallow ($|dy/dx| < 1$), so we step along the x -axis. In Region 2, the curve becomes steep ($|dy/dx| \geq 1$), so stepping in x would leave holes; we must switch to stepping along the y -axis."

Q6: "HOW DOES THE PARITY RULE WORK IN SCANLINE POLYGON FILLING?"

Your Answer: "For every horizontal scanline, we find intersections with polygon boundary edges and sort them from left to right. We toggle our fill state on odd intersections (1st, 3rd) and turn it off on even intersections (2nd, 4th), filling only the interior spans."

Q7: "WHY DID YOU USE INVERSE MAPPING INSTEAD OF FORWARD MAPPING FOR 2D ROTATION?"

Your Answer: "Forward mapping calculates where input pixels go, which causes rounding gaps and unpainted black holes in the destination. Inverse mapping iterates through every destination pixel and samples backward using the inverse rotation matrix, guaranteeing every destination pixel is filled."

Q8: "WHAT IS COYOTE TIME IN GAME DEVELOPMENT?"

Your Answer: "Coyote Time provides a brief grace window (80 milliseconds in our engine) allowing the player to jump right after running off an edge, making the game controls feel forgiving, responsive, and fair."

Q9: "HOW DOES YOUR AUDIO SYNTHESIZER WORK WITHOUT LOADING AUDIO WAV FILES?"

Your Answer: "Our audio engine generates sound procedurally by synthesizing square waves at specific frequencies (e.g. 140 Hz ramping to 420 Hz for jump chirps) directly in memory, mimicking authentic 1980s 8-bit arcade hardware."

Q10: "HOW DOES YOUR CODE RUN ON BOTH WINDOWS PC AND ANDROID PHONES?"

Your Answer: "The project is architected with a decoupled core. The pure raster engine outputs an 800x300 pixel buffer. On PC, Qt presents this buffer to a native desktop window. On Android, the engine is packaged with a hardware-accelerated full-screen shell that runs the exact same raster pipeline and accepts mobile touch events."

